

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

■ TABLE OF CONTENTS

|     |                                           |        |
|-----|-------------------------------------------|--------|
| 1.  | Introduction                              | Part 1 |
| 2.  | Environment Setup                         | Part 1 |
| 3.  | Deploying the Flask Application in Docker | Part 1 |
| 4.  | Configuring Wazuh to Monitor Docker       | Part 1 |
| 5.  | Observing Container Activity in Wazuh     | Part 1 |
| 6.  | Conclusion                                | Part 1 |
| 7.  | Architecture Overview                     | Part 2 |
| 8.  | Section A — App & Logging                 | Part 2 |
| 9.  | Section B — Brute-Force Attack (Hydra)    | Part 2 |
| 10. | Section C — SQL Injection (SQLi)          | Part 2 |
| 11. | Troubleshooting & Challenges              | Part 2 |
| 12. | Reflection & Lessons Learned              | Part 2 |
| 13. | Appendix — Code Snippets                  | Part 2 |

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

## PART 1

### Containerized Flask Deployment & Wazuh Integration

#### ■ 1. INTRODUCTION

This report documents the process of deploying a Flask web application inside a Docker container on a virtual machine and monitoring its activity using the Wazuh security platform. The project demonstrates how containerization improves application deployment flexibility and how Wazuh agents can be configured to track Docker container lifecycle and application-level events for security visibility.

- ◆ Set up the Docker environment and deploy a Flask application in a container.
- ◆ Configure Wazuh agents to monitor container events and application logs.
- ◆ Analyze activity logs through the Wazuh dashboard.
- ◆ Simulate brute-force and SQL injection attacks to test detection capabilities.
- ◆ Demonstrate best practices in securing containerized, cloud-native applications.

#### ■ 2. ENVIRONMENT SETUP

| Component         | Details                                                |
|-------------------|--------------------------------------------------------|
| Operating System  | Ubuntu 22.04.5 LTS (Jammy)                             |
| Docker Version    | Docker 28.5.1, build e180ab8 · Compose v2.40.0         |
| Wazuh Agent       | v4.10.3 — connected to manager, verified via dashboard |
| Flask Application | Python 3.11-slim container, exposed on port 8080→80    |
| Virtual Machine   | AWS EC2 instance — IP 172.31.46.186                    |

#### ■ 3. DEPLOYING THE FLASK APPLICATION IN DOCKER

##### Dockerfile

The following Dockerfile was used to containerize the Flask application:

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
COPY users.db .
RUN pip install flask
EXPOSE 80
CMD ["python", "app.py"]
```

## Build & Run Commands

```
docker build -t flask-app .
```

Creates a Docker image from the Dockerfile, named 'flask-app'.

```
docker run -d -p 8080:80 flask-app
```

Runs the container in detached mode, mapping host port 8080 → container port 80.

```
docker ps
```

Verifies the container is running and shows port mappings.

## ■ 4. CONFIGURING WAZUH TO MONITOR DOCKER

Wazuh was configured to monitor Docker container events by enabling the **docker-listener** wodle in the agent configuration file. This allows Wazuh to track container lifecycle events including creation, start, stop, kill, and network changes.

```
<wodle name="docker-listener">
  <disabled>no</disabled>
  <interval>10m</interval>
  <attempts>5</attempts>
  <run_on_start>no</run_on_start>
</wodle>
```

## Events Observed in Wazuh Dashboard

Container Created

Container Started

Image Created &  
Tagged

Network Created/Di  
sconnected

Container  
Stopped/Killed

Container  
Restarted

## ■ 5. OBSERVING CONTAINER ACTIVITY IN WAZUH

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

Container lifecycle events were tested using the following commands. Each action was recorded and visible in the Wazuh security events dashboard, demonstrating real-time visibility into containerized workloads.

```
$ sudo docker restart flask-app
```

```
$ sudo docker stop flask-app
```

```
$ sudo docker start flask-app
```

HTTP interactions with the Flask app (login attempts, page requests) were also captured by Wazuh, showing the full HTTP request lifecycle. Rule IDs assigned: **100001** (auth failure), **100002** (login success), **100003** (invalid login attempt).

## ■ 6. CONCLUSION — PART 1

- ◆ **Container Monitoring is Essential:** Containers run with shared resources and network connectivity, making them high-risk if unmonitored. Wazuh fills this gap with real-time event collection.
- ◆ **Project Foundation for Security Automation:** By integrating Wazuh with Docker, this project establishes a scalable, automated foundation for security alerting, anomaly detection, and incident response in containerized environments.
- ◆ **Practical Security Visibility:** Container start, stop, kill, and recreate events are all visible in the Wazuh dashboard, enabling rapid detection of unauthorized container manipulation.

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

## PART 2

### Attack Simulation, Detection & Log Analysis

#### 7. ARCHITECTURE OVERVIEW

The system is composed of four layers working together to provide real-time security monitoring of the containerized Flask application:

| Layer | Component                     | Role                                                                                  |
|-------|-------------------------------|---------------------------------------------------------------------------------------|
| 1     | App Container (Web/App)       | Runs Flask app inside Docker. Generates all user events: registration, login, errors. |
| 2     | Log Destination (File/Syslog) | Stores logs generated by the application in JSON format, ready for Wazuh collection.  |
| 3     | Wazuh Agent                   | Collects logs, applies decoders and rules, forwards alerts to the manager.            |
| 4     | Wazuh Manager / Dashboard     | Receives alerts, displays user activity, detects brute-force and SQLi attacks.        |

#### 8. SECTION A — APP & LOGGING

Application logs are generated in **JSON format**, enabling Wazuh to efficiently parse events and trigger targeted alerts. Each log entry captures the following fields:

| Field     | Description                    | Example                       |
|-----------|--------------------------------|-------------------------------|
| timestamp | Event time                     | e.g. 2025-10-27T20:21:00.676Z |
| user      | Username performing the action | e.g. group4, admin            |
| action    | Type of action                 | login, insert, update, delete |

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

|        |                         |                                |
|--------|-------------------------|--------------------------------|
| status | Outcome                 | success, invalid login attempt |
| src_ip | Source IP address       | e.g. 172.31.39.194             |
| table  | Database table affected | users (for DB operations)      |

■ Rationale: Structured JSON logs allow Wazuh to detect events efficiently — successful/failed logins, INSERT/UPDATE/DELETE database operations — and provide traceable, readable records for auditing.

## ■ 9. SECTION B — BRUTE-FORCE ATTACK (HYDRA)

### ATTACK SIMULATION

**Tool:** Hydra · **Target:** Flask login page · **Username:** hacker · **Wordlist:** passwords.txt  
**Start:** 2025-11-04 00:53:53 · **End:** 2025-11-04 00:53:55 · **Result:** Password found — **liverpool**

- ◆ **Attack Scenario:** Hydra automated HTTP POST login attempts against the Flask login page, cycling through a password wordlist for username 'hacker'.
- ◆ **Process:** Each attempt sent a new password from the list. Hydra found the correct password (**liverpool**) in approximately 2 seconds — 16 tasks, 111 login tries.
- ◆ **Detection:** Wazuh monitored application logs and detected a high volume of failed login attempts from a single IP in a short period — a classic brute-force signature.
- ◆ **Alert Generated:** Wazuh Rule **100001** (auth failure from same IP) triggered, logging the username, attempt count, and source IP address.

## ■ 10. SECTION C — SQL INJECTION (SQLI)

### SQL INJECTION

**Tools:** curl, sqlmap · **Target:** /vulLogin endpoint  
**Payload:** username=hacker&password=' OR '1'='1  
**Result:** Authentication bypassed — **Welcome, hacker!**

- ◆ **Scenario:** An SQL injection was manually performed against the login form. The application built SQL queries from raw user input without sanitization.
- ◆ **Payload:** Entering admin' OR '1'='1' -- as the username changed the query logic, making the condition always true and bypassing authentication.
- ◆ **sqlmap Testing:** sqlmap was also used to automate SQLi detection, confirming the POST parameter 'password' was injectable.
- ◆ **Detection:** Wazuh Rule **100011** (SQL Injection detected) and Rule **100012** (Database error — possible SQLi or malformed query) were triggered.

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

- ◆ **Defense Analysis:** Application logs were monitored in real time using `sudo tail -f app.log`, revealing injected SQL queries.

| Rule ID | Description                    | Level | Groups                              |
|---------|--------------------------------|-------|-------------------------------------|
| 100011  | SQL Injection detected         | 9     | web_attack, sqli, flask_sql_attempt |
| 100012  | Database error (possible SQLi) | 7     | web_attack, sqli, flask_db_error    |
| 100001  | Auth failure from same IP      | 5     | authentication_failed, local        |
| 100002  | User successfully logged in    | 3     | flask_login_success                 |
| 100003  | Invalid login attempt          | 6     | flask_login_failure                 |

## 11. TROUBLESHOOTING & CHALLENGES

### Log file permission problems

■ **Problem:** Docker containers sometimes restrict file access, causing missing or incomplete log data for Wazuh.

✓ **Solution:** Adjusted Docker volume and file permissions so both Flask and Wazuh could reliably write and read logs.

### Decoder misconfiguration in Wazuh

■ **Problem:** Wazuh occasionally failed to parse custom JSON logs, reducing detection accuracy for security events.

✓ **Solution:** Rewrote Wazuh decoder settings to exactly match the log format, greatly improving attack detection accuracy.

### Excessive noisy alerts

■ **Problem:** Overly broad rules triggered overwhelming numbers of alerts, making it hard to focus on real threats.

✓ **Solution:** Fine-tuned Wazuh rule sets to suppress false positives and focus alerting on serious security incidents.

## 12. REFLECTION & LESSONS LEARNED

- ◆ **Structured logging is essential:** Clear, machine-readable JSON logs make monitoring and incident response faster and more reliable.
- ◆ **Correlation rules reduce noise:** Well-tuned Wazuh rules help security teams focus on genuine threats rather than chasing false positives.

# Hosting and Monitoring a Containerized Flask Application

Using Docker and Wazuh for Enhanced Cloud-Native Security · Parts 1 & 2

Course: 420-1C1-DW · Preventative Monitoring · Instructor: Nneamaka Chrysa Adirika

Students: Yinghai Deng, Mohammad Salem Hassani · Date: Oct–Nov 2025

Docker

Wazuh SIEM

Brute-Force

SQLi

Cloud-Native Security

- ◆ **Secure coding matters:** Input validation and parameterized queries are the primary defense against SQL injection — detection alone is not enough.
- ◆ **Realistic attack simulations build skills:** Hands-on testing with Hydra and sqlmap deepens understanding of real-world attack patterns and SOC workflows.
- ◆ **Container monitoring is different:** Short-lived containers require continuous, automated monitoring — traditional static security measures are insufficient.

## ■ 13. APPENDIX — CODE SNIPPETS

### Python Logging Configuration

```
import logging
logging.basicConfig(
    format='%(asctime)s event=%(event)s user=%(user)s result=%(result)s',
    level=logging.INFO
)
logging.info(' ', extra={'event': 'login', 'user': 'mohammad', 'result': 'failure'})
```

### Wazuh Decoder (XML)

```
<decoder name="flask_app">
  <program_name>flask-app</program_name>
  <regex>timestamp=(?P<timestamp>\S+) event=(?P<event>\S+) user=(?P<user>\S+)
result=(?P<result>\S+)</regex>
</decoder>
```

### Wazuh Rule (XML)

```
<rule id="100001" level="10">
  <decoded_as>flask_app</decoded_as>
  <field name="event">login</field>
  <field name="result">failure</field>
  <description>Failed login detected</description>
</rule>
```